JAVA PROGRAMMING

Practice Program Unit 4

Submitted by:

Faustena S

(2548313)

2 MDS A

Faculty:

Dr. Heamenth K S

Submitted on 03 Feb. 26

Department of Computer Science and Statistics

Christ (Deemed To Be University),

Yeswanthpur, Bangalore

Defining Classes and Objects

1. Student class with one object

```
class Student {
    String name;
    int age;
    String course;
}

public class lab2 {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Tina ";
        s1.age = 20;
        s1.course = "Data Science";

        System.out.println("Name: " + s1.name);
        System.out.println("Age: " + s1.age);
        System.out.println("Course: " + s1.course);
    }
}
```

```
PROBLEMS 12   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS E:\Tina DeskTop\Java>  & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-agentlib:jdwp=tr
st:54857' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\use
55c2760b8d783903cbfe4a5de\redhat.java\jdt_ws\Java_62d56fb3\bin' 'lab2'
Name: Tina
Age: 20
Course: Data Science
PS E:\Tina DeskTop\Java>
```

2. Two student objects with method

```
class Student {
    String name;
    String course;
}

class PostgraduateStudent extends Student {
    String specialization;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Specialization: " + specialization);
    }
}

public class p2 {
    public static void main(String[] args) {
        PostgraduateStudent pg = new PostgraduateStudent();
```

```
        pg.name = "Tina";
        pg.course = "MDS";
        pg.specialization = "AI/ML";

        pg.display();
    }
}
```

```
PROBLEMS 12    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS E:\Tina DeskTop\Java>  & 'C:\Program Files\Java\jdk-25\bin\jav
\Users\user\AppData\Roaming\Code\User\workspaceStorage\637b3b355c
Name: Tina
Course: MDS
Specialization: AI/ML
PS E:\Tina DeskTop\Java>
```

3. Department class with students

```
class Student1 {
    String name;
    int age;
    String course;

    void displayAdmission() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println();
    }
}
```

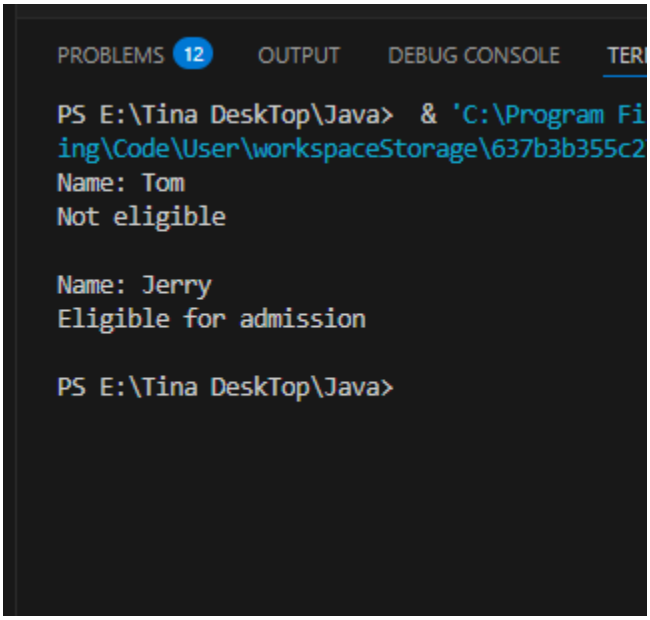
```
public class p3 {
    public static void main(String[] args) {
        Student1 s1 = new Student1();
        s1.name = "Tina";
        s1.age = 19;
        s1.course = "Data Science";

        Student1 s2 = new Student1();
        s2.name = "Sruthi";
        s2.age = 21;
        s2.course = "Maths";

        s1.displayAdmission();
        s2.displayAdmission();
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\Program File
ing\Code\User\workspaceStorage\637b3b355c276
Name: Tina
Age: 19
Course: Data Science

Name: Sruthi
Age: 21
Course: Maths

PS E:\Tina DeskTop\Java>
```

4. Age eligibility check

```
class Student {
    String name;
    int age;

    void checkEligibility() {
        System.out.println("Name: " + name);
        if(age >= 18) {
            System.out.println("Eligible for admission");
        } else {
            System.out.println("Not eligible");
        }
        System.out.println();
    }
}

public class Test {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Tom";
        s1.age = 17;

        Student s2 = new Student();
        s2.name = "Jerry";
        s2.age = 19;

        s1.checkEligibility();
        s2.checkEligibility();
```

```
    }
}
```



5. Array of students

```
class Student4 {
    String name;
    int age;
    boolean admitted;

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class p6 {
    public static void main(String[] args) {
        Student4[] students = new Student4[3];
```

```
        students[0] = new Student4();
        students[0].name = "tina";
        students[0].age = 20;
        students[0].admitted = true;

        students[1] = new Student4();
        students[1].name = "bolu";
        students[1].age = 19;
        students[1].admitted = false;

        students[2] = new Student4();
        students[2].name = "dolu";
        students[2].age = 21;
        students[2].admitted = true;

        System.out.println("Admitted Students:");
        for(int i = 0; i < students.length; i++) {
            if(students[i].admitted) {
                students[i].display();
            }
        }
    }
}
```

```
PS E:\Tina DeskTop\Java>
PS E:\Tina DeskTop\Java>  e:; cd 'e:\Tina Desk
cp' 'C:\Users\user\AppData\Roaming\Code\User\w
Admitted Students:
Name: tina, Age: 20
Name: dolu, Age: 21
PS E:\Tina DeskTop\Java>
```

Constructors and Methods

1. Constructor with name and course

```
class Student5 {
    String name;
    String course;

    Student5(String n, String c) {
        name = n;
        course = c;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
    }
}

public class p7 {
    public static void main(String[] args) {
        Student5 s1 = new Student5("Jo jo", "CSE");
        s1.display();
    }
```

```
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\
iew' '-XX:+ShowCodeDetailsInExce
a_62d56fb3\bin' 'p7'
Name: Jo jo
Course: CSE
PS E:\Tina DeskTop\Java>
```

2. Auto-generate student ID

```
class Student6 {
    String name;
    int studentId;
    static int counter = 1000;

    Student6(String n) {
        name = n;
        studentId = ++counter;
    }

    void display() {
        System.out.println("ID: " + studentId + ", Name: " + name);
    }
}

public class p9 {
    public static void main(String[] args) {
        Student6 s1 = new Student6("tina");
        Student6 s2 = new Student6("terasia");
        Student6 s3 = new Student6("Carlin");
```

```
        s1.display();
        s2.display();
        s3.display();
    }
}
```

3. Overloaded constructors

```
class Student {
    String name;
    String course;
    boolean hostel;

    Student(String n, String c) {
        name = n;
        course = c;
        hostel = false;
    }

    Student(String n, String c, boolean h) {
        name = n;
        course = c;
        hostel = h;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
```

```
        System.out.println("Hostel: " + hostel);
        System.out.println();
    }
}

public class Test {
    public static void main(String[] args) {
        Student s1 = new Student("John", "CSE");
        Student s2 = new Student("Mary", "ECE", true);

        s1.display();
        s2.display();
    }
}
```

```
PROBLEMS 12    OUTPUT    DEBUG CONSOLE    TERMINAL

PS E:\Tina DeskTop\Java>  & 'C:\Program Files\Java
ing\Code\User\workspaceStorage\637b3b355c2760b8d78
Student: John
Department: CSE

Student: Mary
Department: ECE

PS E:\Tina DeskTop\Java>
```

```
class Student7 {
    String name;
    String course;
    boolean hostel;

    Student7(String n, String c) {
        name = n;
        course = c;
        hostel = false;
    }

    Student7(String n, String c, boolean h) {
        name = n;
        course = c;
        hostel = h;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Hostel: " + hostel);
        System.out.println();
    }
}

public class p10 {
    public static void main(String[] args) {
```

```
        Student7 s1 = new Student7("tina", "CSE");
        Student7 s2 = new Student7("Money", "ECE", true);

        s1.display();
        s2.display();
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\Program
ing\Code\User\workspaceStorage\637b3b35
Name: tina
Course: CSE
Hostel: false

Name: Money
Course: ECE
Hostel: true

PS E:\Tina DeskTop\Java>
```

4. Methods to display and update

```
class Student8 {
    String name;
    String course;

    Student8(String n, String c) {
        name = n;
        course = c;
    }
```

```
    void displayDetails() {
        System.out.println("Name: " + name + ", Course: " + course);
    }

    void updateCourse(String newCourse) {
        course = newCourse;
        System.out.println("Course updated to: " + course);
    }
}

public class p11 {
    public static void main(String[] args) {
        Student8 s1 = new Student8("Tom", "Mathematics");
        s1.displayDetails();
        s1.updateCourse("Physics");
        s1.displayDetails();
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\Program File
iew' '-XX:+ShowCodeDetailsInExceptionMessage
a_62d56fb3\bin' 'p11'
Name: Tom, Course: Mathematics
Course updated to: Physics
Name: Tom, Course: Physics
PS E:\Tina DeskTop\Java>
```

5. Constructor then method call

```
class Student9 {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class UndergraduateStudent extends Student9 {
    String program;

    void showProgram() {
        System.out.println("Program: " + program);
    }
}

public class p12 {
    public static void main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent();
        ug.name = "Tina";
        ug.age = 20;
        ug.program = "B.Tech";

        ug.display();
        ug.showProgram();
```

```
    }
}
```

```
PS E:\Tina DeskTop\Java>  & 'C:\Progr
iew' '-XX:+ShowCodeDetailsInException
a_62d56fb3\bin' 'p12'
Name: Tina
Age: 20
Program: B.Tech
PS E:\Tina DeskTop\Java>
```

Inheritance

1. Base and derived class

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

class UndergraduateStudent extends Student {
    String program;

    void showProgram() {
        System.out.println("Program: " + program);
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent();
        ug.name = "Alice";
        ug.age = 19;
        ug.program = "B.Tech";

        ug.display();
        ug.showProgram();
    }
}
```

2. PostgraduateStudent with specialization

```
class Student {
    String name;
    String course;
}

class PostgraduateStudent extends Student {
    String specialization;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Specialization: " + specialization);
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        PostgraduateStudent pg = new PostgraduateStudent();
        pg.name = "Bob";
        pg.course = "M.Tech";
        pg.specialization = "AI/ML";

        pg.display();
    }
}
```

3. InternationalStudent with passport

```
class Student {
    String name;
    String course;
}

class InternationalStudent extends Student {
    String passportNo;
    String country;

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Passport: " + passportNo);
        System.out.println("Country: " + country);
    }
```

```
}

public class Test {
    public static void main(String[] args) {
        InternationalStudent is = new InternationalStudent();
        is.name = "John";
        is.course = "CSE";
        is.passportNo = "P123456";
        is.country = "USA";

        is.display();
    }
}
```

4. Super keyword demo

```
class Student {
    String name;

    Student(String n) {
        name = n;
        System.out.println("Parent constructor");
    }
}

class UndergraduateStudent extends Student {
    String program;

    UndergraduateStudent(String n, String p) {
```

```
        super(n);
        program = p;
        System.out.println("Child constructor");
    }

    void display() {
        System.out.println("Name: " + name + ", Program: " + program);
    }
}

public class Test {
    public static void main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent("Alice", "B.Tech");
        ug.display();
    }
}
```

5. Code reuse using inheritance

```
class Student {
    String name;
    int rollNo;

    void setBasicDetails(String n, int r) {
        name = n;
        rollNo = r;
    }

    void displayBasic() {
```

```
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}

class UndergraduateStudent extends Student {
    String branch;

    void setBranch(String b) {
        branch = b;
    }

    void displayAll() {
        displayBasic();
        System.out.println("Branch: " + branch);
    }
}

public class Test {
    public static void main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent();
        ug.setBasicDetails("Tom", 101);
        ug.setBranch("CSE");
        ug.displayAll();
    }
}
```

Method Overriding

1. Student class with calculateFee

```
class Student {
    String name;

    int calculateFee() {
        return 10000;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println("Fee: " + s.calculateFee());
    }
}
```

2. Override in UndergraduateStudent

```
class Student {
    int calculateFee() {
        return 10000;
    }
}

class UndergraduateStudent extends Student {
    int calculateFee() {
        return 15000;
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        UndergraduateStudent ug = new UndergraduateStudent();
        System.out.println("UG Fee: " + ug.calculateFee());
    }
}
```

3. Override in PostgraduateStudent

```
class Student {
    int calculateFee() {
        return 10000;
    }
}

class PostgraduateStudent extends Student {
    int calculateFee() {
        return 20000;
    }
}

public class Test {
    public static void main(String[] args) {
        PostgraduateStudent pg = new PostgraduateStudent();
        System.out.println("PG Fee: " + pg.calculateFee());
    }
}
```

4. Parent reference

```
class Student {
    int calculateFee() {
        return 10000;
    }
}

class UndergraduateStudent extends Student {
    int calculateFee() {
        return 15000;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new UndergraduateStudent();
        System.out.println("Fee: " + s.calculateFee());
    }
}
```

5. Runtime behavior

```
class Student {
    int calculateFee() {
        return 10000;
    }
}

class UndergraduateStudent extends Student {
    int calculateFee() {
```

```
        return 15000;
    }
}

class PostgraduateStudent extends Student {
    int calculateFee() {
        return 20000;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s1 = new UndergraduateStudent();
        Student s2 = new PostgraduateStudent();

        System.out.println("UG Fee: " + s1.calculateFee());
        System.out.println("PG Fee: " + s2.calculateFee());
    }
}
```

Polymorphism

1. Method overloading

```
class Admission {
    void admitStudent(String name) {
        System.out.println("Admitted: " + name);
    }

    void admitStudent(String name, String course) {
```

```
        System.out.println("Admitted: " + name + " in " + course);
    }

    void admitStudent(String name, String course, int year) {
        System.out.println("Admitted: " + name + " in " + course + " year " + year);
    }
}

public class Test {
    public static void main(String[] args) {
        Admission a = new Admission();
        a.admitStudent("John");
        a.admitStudent("Mary", "CSE");
        a.admitStudent("Tom", "ECE", 2);
    }
}
```

2. Overload calculateFee

```
class Student {
    int calculateFee(String course) {
        return 10000;
    }

    int calculateFee(String course, boolean hostel) {
        if(hostel)
            return 15000;
        return 10000;
    }
```

```
    int calculateFee(String course, boolean hostel, boolean transport) {
        int fee = 10000;
        if(hostel) fee += 5000;
        if(transport) fee += 2000;
        return fee;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println("Basic: " + s.calculateFee("CSE"));
        System.out.println("With hostel: " + s.calculateFee("CSE", true));
        System.out.println("Full: " + s.calculateFee("CSE", true, true));
    }
}
```

3. Runtime polymorphism

```
class Student {
    void display() {
        System.out.println("Student");
    }
}

class UndergraduateStudent extends Student {
    void display() {
        System.out.println("Undergraduate Student");
```

```
    }
}

class PostgraduateStudent extends Student {
    void display() {
        System.out.println("Postgraduate Student");
    }
}

public class Test {
    public static void main(String[] args) {
        Student s1 = new UndergraduateStudent();
        Student s2 = new PostgraduateStudent();

        s1.display();
        s2.display();
    }
}
```

4. Dynamic method dispatch

```
class Student {
    int calculateFee() {
        return 10000;
    }
}

class UndergraduateStudent extends Student {
    int calculateFee() {
```

```
        return 15000;
    }
}

class PostgraduateStudent extends Student {
    int calculateFee() {
        return 20000;
    }
}

public class Test {
    static void processFee(Student s) {
        System.out.println("Fee: " + s.calculateFee());
    }

    public static void main(String[] args) {
        processFee(new UndergraduateStudent());
        processFee(new PostgraduateStudent());
    }
}
```

5. Array of Student references

```
class Student {
    String name;
    int calculateFee() {
        return 10000;
    }
}
```

```
class UndergraduateStudent extends Student {
    int calculateFee() {
        return 15000;
    }
}

class PostgraduateStudent extends Student {
    int calculateFee() {
        return 20000;
    }
}

public class Test {
    public static void main(String[] args) {
        Student[] students = new Student[3];

        students[0] = new Student();
        students[1] = new UndergraduateStudent();
        students[2] = new PostgraduateStudent();

        for(int i = 0; i < students.length; i++) {
            System.out.println("Fee: " + students[i].calculateFee());
        }
    }
}
```

Encapsulation and Abstraction

1. Getters and setters

```
class Student {
    private String name;
    private int age;

    public void setName(String n) {
        name = n;
    }

    public String getName() {
        return name;
    }

    public void setAge(int a) {
        age = a;
    }

    public int getAge() {
        return age;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.setName("John");
        s.setAge(20);
```

```
        System.out.println("Name: " + s.getName());
        System.out.println("Age: " + s.getAge());
    }
}
```

2. Restrict admission number

```
class Student {
    private int admissionNo;
    private String name;

    public Student(int admNo, String n) {
        admissionNo = admNo;
        name = n;
    }

    public int getAdmissionNo() {
        return admissionNo;
    }

    public void display() {
        System.out.println("Admission No: " + admissionNo);
        System.out.println("Name: " + name);
    }
}

public class Test {
    public static void main(String[] args) {
```

```
        Student s = new Student(101, "Alice");
        s.display();
        System.out.println("Admission: " + s.getAdmissionNo());
    }
}
```

3. Validate age

```
class Student {
    private String name;
    private int age;

    public void setAge(int a) {
        if(a >= 18 && a <= 25) {
            age = a;
            System.out.println("Age set successfully");
        } else {
            System.out.println("Invalid age");
        }
    }

    public int getAge() {
        return age;
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
```

```
        s.setAge(17);
        s.setAge(20);
        System.out.println("Age: " + s.getAge());
    }
}
```

4. Abstract class

```
abstract class StudentAdmission {
    String name;

    abstract int calculateFee();

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Fee: " + calculateFee());
    }
}

class UndergraduateStudent extends StudentAdmission {
    int calculateFee() {
        return 15000;
    }
}

class PostgraduateStudent extends StudentAdmission {
    int calculateFee() {
        return 20000;
    }
```

```
}

public class Test {
    public static void main(String[] args) {
        StudentAdmission s1 = new UndergraduateStudent();
        s1.name = "John";
        s1.display();

        StudentAdmission s2 = new PostgraduateStudent();
        s2.name = "Mary";
        s2.display();
    }
}
```

5. Interface implementation

```
interface ScholarshipEligible {
    boolean checkEligibility();
}

class Student implements ScholarshipEligible {
    String name;
    float marks;

    public boolean checkEligibility() {
        return marks >= 85.0;
    }

    void display() {
```

```
        System.out.println("Name: " + name);
        System.out.println("Eligible: " + checkEligibility());
    }
}

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.marks = 90.5f;
        s.display();
    }
}
```

Packages

1. Package university.admission

File: university/admission/Admission.java

```
package university.admission;

public class Admission {
    public void admitStudent(String name) {
        System.out.println("Admitted: " + name);
    }
}
```

File: Test.java

```
import university.admission.Admission;

public class Test {
```

```
    public static void main(String[] args) {
        Admission a = new Admission();
        a.admitStudent("John");
    }
}
```

2. Package university.students

File: university/students/Student.java

```
package university.students;

public class Student {
    public String name;
    public int rollNo;

    public void display() {
        System.out.println("Name: " + name + ", Roll: " + rollNo);
    }
}
```

File: Test.java

```
import university.students.Student;

public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.rollNo = 101;
        s.display();
    }
```

```
}
```

3. Access from another package

File: university/students/Student.java

```
package university.students;

public class Student {
    public String name;

    public void display() {
        System.out.println("Student: " + name);
    }
}
```

File: university/admission/Admission.java

```
package university.admission;

import university.students.Student;

public class Admission {
    public void processAdmission() {
        Student s = new Student();
        s.name = "Bob";
        s.display();
    }
}
```

4. Protected access

File: university/students/Student.java

```
package university.students;
```

```
public class Student {
    protected String name;

    protected void showName() {
        System.out.println("Name: " + name);
    }
}
```

File: university/admission/UGStudent.java

```
package university.admission;

import university.students.Student;

public class UGStudent extends Student {
    public void display() {
        name = "Charlie";
        showName();
    }
}
```

5. Mini university system

File: university/students/Student.java

```
package university.students;

public class Student {
    public String name;
    public int rollNo;
}
```

File: university/admission/Admission.java

```
package university.admission;

import university.students.Student;

public class Admission {
    public Student admitStudent(String name, int roll) {
        Student s = new Student();
        s.name = name;
        s.rollNo = roll;
        return s;
    }
}
```

File: university/fees/FeeCalculator.java

```
package university.fees;

import university.students.Student;

public class FeeCalculator {
    public int calculateFee(Student s) {
        return 15000;
    }
}
```

File: Test.java

```
import university.students.Student;
import university.admission.Admission;
import university.fees.FeeCalculator;
```

```
public class Test {
    public static void main(String[] args) {
        Admission adm = new Admission();
        Student s = adm.admitStudent("John", 101);

        FeeCalculator fc = new FeeCalculator();
        int fee = fc.calculateFee(s);

        System.out.println("Student: " + s.name);
        System.out.println("Roll: " + s.rollNo);
        System.out.println("Fee: " + fee);
    }
}
```